Expected vs. Abnormal Execution Behavior

- A program's execution should follow some **expected** behavior by its developers
  - Expected control/data flow
  - Expected access-control policy
    - E.g., admin can do this; normal users can do that
- However, an attacker feeds the program a malicious input and induces **abnormal** execution behavior
  - Destroying the program's integrity during execution
  - E.g., make a return to target an unintended address

Enforcing Execution Integrity

- Idea
  - Statically compute the program's expected behavior
  - Dynamically check if the program follows the expected behavior using a reference monitor
  - If checking fails, stop the program from execution
- SFI follows this pattern
  - We expect the program's memory access stay within the SFI sandbox

Kinds of Execution Integrity

- **Control-Flow Integrity**
  - A program's control flow should follow expected control flow
- **Memory Safety**
  - A program should access memory within buffer bounds and during its lifetime
- Data-flow integrity
- …

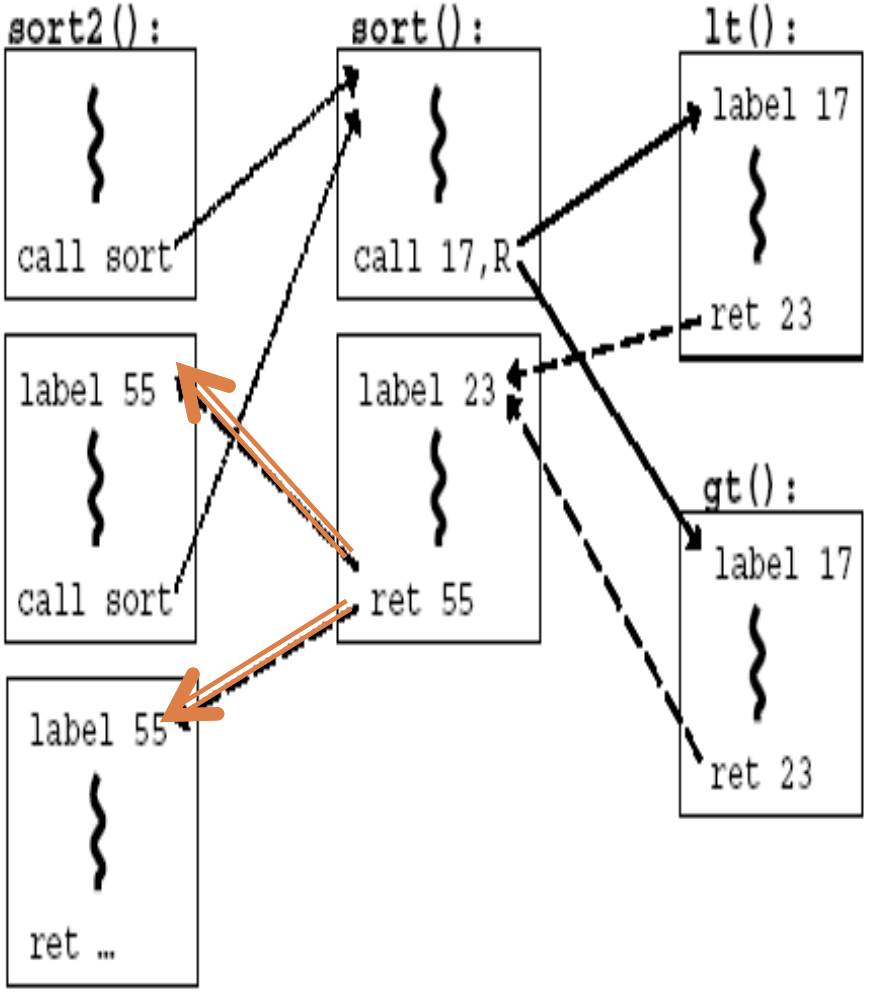
Control-Flow Integrity (CFI)

Control Flow Graph (CFG)

- CFG is a graph G=(V,E)
  - V is a set of nodes; each represents an instruction (or a basic block of instructions)
  - E is a set of control-flow edges; edge (n1,n2) means that n2 can succeed n1 in some execution
- A CFG of a program encodes its **expected control flow**
- How to get the CFG?
  - Static analysis of source/binary code; Execution profiling; Explicit specification

CFG Example with Indirect Branches


```
bool lt(int x, int y) {return x<y;}
bool gt(int x, int y) {return x>y;}
void sort(…) {…; return;}
void sort2(int a[], int b[], int len) {
    sort(a, len, lt);
    sort(b, len, gt);
}
```



Main Idea of Control-Flow Integrity

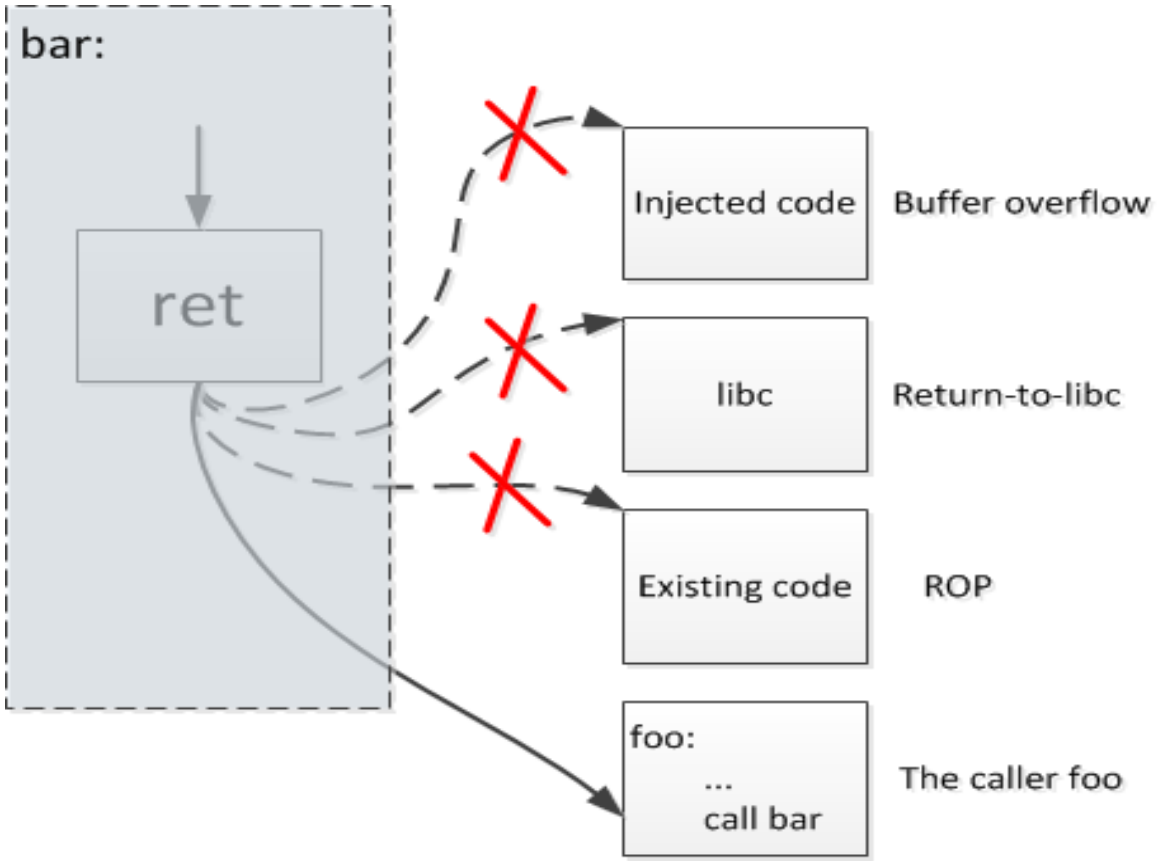
1) Pre-determine the control flow graph (CFG) of an application
2) Enforce the CFG through a binary-level IRM

CFI Policy: execution must follow the pre-determined control flow graph, even under attacks

Attack model: the attacker can change memory between instructions, but cannot directly change contents in registers

CFI Prevents Control-Flow Hijacking

Lots of attacks induce illegal control-flow transfers: buffer overflow, return-to-libc, ROP



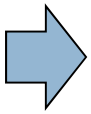
CFI Enforcement

- Can be enforced through an Inline Reference Monitor [Abadi, Budiu, Erlingsson, Ligatti CCS 2005]
- For computed jumps (returns, indirect calls/jumps)
  - Insert an ID at every destination given by the CFG
  - Insert a runtime check to compare whether the ID of the target instruction matches the expected ID
- A direct jump can be checked statically

CFI Example I

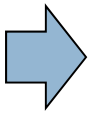
Any side-effect free instruction with an ID embedded would do

```
call sort
```



```
call sor
prefetchnta $ID
```

```
sort:
  …
  ret
```

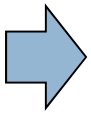


```
sort:
  …
  ecx := mem(esp)
  esp := esp + 4
  if mem(ecx+3) <> $ID goto error
  jmp ecx
```

Opcode of prefetch takes 3 bytes

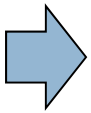
CFI Example II


```
call sort
...
call sort
```



```
call sort
prefetchnta $ID
...
call sort
prefetchnta $ID
```

```
sort:
  ...
  ret
```



```
sort:
  ...
  ecx := mem(esp)
  esp := esp + 4
  if mem(ecx+3) <> $ID goto error
  jmp ecx
```

Allow returning to either of the call sites

CFI Assumptions

- Non-writable code region
  - IDs are embedded into the code
- Non-executable data region
  - Otherwise, the attacker can fake an ID
- Unique IDs
  - Bit patterns chosen as IDs must not appear anywhere else in the code region

CFG is an Overapproximation

- A CFG is sound as long as it over-approximates all possible runtime control flows
  - The same program can have multiple CFGs
  - Different over-approximations result in the CFGs of different precision
  - Some coarse grained and some fine grained

CFG Overapproximation Examples

- An indirect call must target the beginning of a function
  - Called coarse-grained CFI
- An indirect call through a function pointer must target a function of a compatible type [MCFI PLDI '14]
  - E.g., int (*fp)(char*, int) can be used to call a function f only if its signature is "int f (char*, int)"
  - Challenges: type casts; the void type sometimes used as a polymorphic type
- Pointer analysis that tracks function pointer creations and uses
  - e.g., taint-based CFI [IEEE Euro S&P '16]

Overapproximation Causes Imprecision

- There are multiple sources of imprecision
- One source: CFG may include unnecessary edges
  - E.g., during CFG construction, the following call may be allowed to call any function of type “int->int”

    fp = &foo;

    …

    call *fp

  Even though in real exactions it can target only foo

Imprecision: Call/Return Mismatch

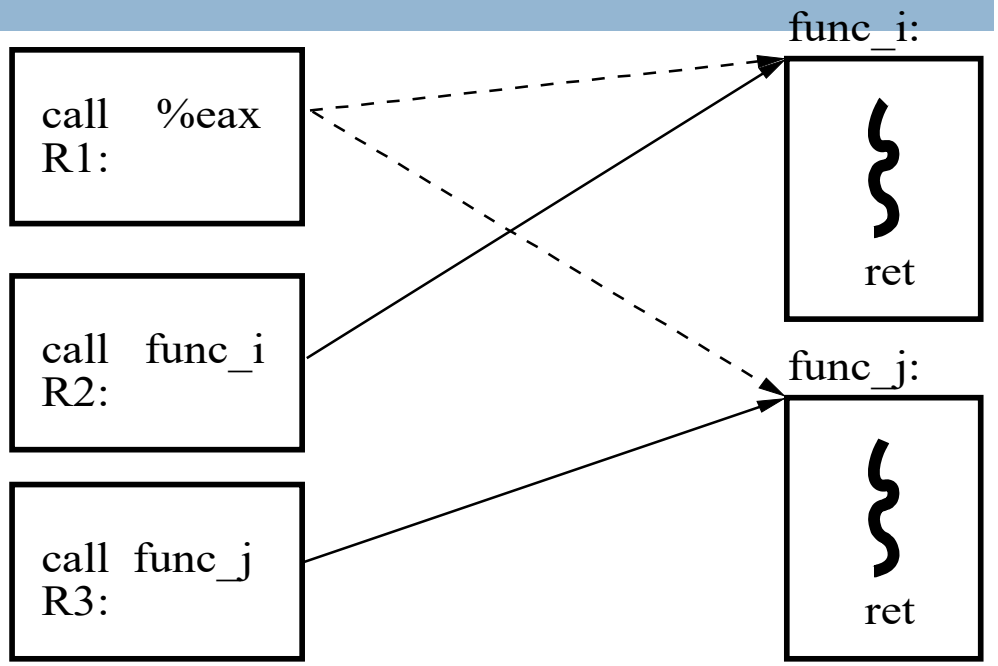

```
void foo1 () {
   …; bar(); …
}
void foo2 () {
   …; bar(); …
}
```

```
void bar () {
  …; return;
}
```

- Return in bar() can return to either foo1 or foo2
- Essentially, pure CFI allows unmatched calls and returns
  - foo1 -> bar -> return to foo2
- It enforces a finite-state machine, instead of pushdown machine

Imprecision: Destination Equivalence

- The ID-based CFI enforcement requires a notion of equivalent destinations
  - Two destinations are equivalent if CFG contains edges to each from the same source
  - Use same ID for equivalent destinations



In the above example, same ID at R1, R2, and R3; then func_j is allowed to return to R2

CFI and Security

- Effective against attacks based on illegal control-flow transfer
  - Stack-based buffer overflow, return-to-libc exploits, pointer subterfuge
- Does **not** protect against attacks that do not violate the program's original CFG
  - Attacks exploiting CFI imprecision
  - Incorrect arguments to system calls
  - Substitution of file names
  - Non-control data attacks

Shadow Stack: Matching Calls and Returns

- On call
  - Push return address on the regular stack
  - Also, push the return address on the shadow stack
- On return
  - Validate the return address on the regular stack with the return address on the shadow stack
- Also, protect the shadow stack so that the program cannot modify it directly
  - E.g., if the program is in user space, put the shadow stack in the kernel space
  - E.g., insert SFI-style checks before memory writes so that writes cannot target the shadow stack memory

Shadow Stack

- Intel Control-Flow Enforcement Technology (CET)
  - Has been announced
  - Not in products yet
- Goal is to enforce shadow stack in hardware
  - Throw an exception when a return does not correspond to a call site
- Challenge: Unconventional control flow
  - There are cases where call-return does not match
  - E.g., Tail calls, setjmp/longjmp, …

Memory Safety

* Some slides borrowed from Dr. Trent Jaeger

Memory Safety

- Memory buffers are allocated and deallocated during program execution
- Each buffer occupies a contiguous range of memory addresses and also has a lifetime
  - Bounds: the lower and upper addresses of the buffer
  - Lifetime: when the buffer is valid for use
    - E.g., a buffer allocated by a function's stack has a lifetime when the function executes; should not be used after the function returns
    - E.g., a buffer that was created by malloc should not be accessed after being freed

Memory Safety: Expected vs. Abnormal Behavior

- **Expected** behavior**:** a buffer should be accessed within its bounds and only during its lifetime
  - **Spatial** memory safety: a buffer should be accessed within its bounds
  - **Temporal** memory safety: a buffer can be accessed only during its lifetime
- **Abnormal** behavior
  - When spatial memory safety is violated, we have buffer overread/overwrite
  - When temporal memory safety is violated, we have things like use-after-free situations

Safe vs. Unsafe Languages

- Some programming languages are memory safe by design
  - Java, Python, C#, Ruby, Scala, Rust, …
  - Via a strong type system or runtime checks
- Memory unsafe languages: C, C++, Objective C
  - The root of many security problems

Enforcing Memory Safety in Unsafe Languages by Reference Monitoring

- General idea: check every memory access to ensure
  - The access is within bounds
  - The access is to a valid object according to its lifetime
- Challenges
  - C/C++ does not track bounds and lifetime of memory objects
    - Additional instrumentation is needed to track that information for performing checks
  - Performance overhead when checking every memory access

Bounds Checks for Spatial Safety

- Goal: prevent buffer overflows
- Basic approach
  - Instrument the program to insert bounds checks

```
int a[100];
…
a[i]=3; //need bounds check: a <= a+i < a + 100
```

How to Get the Bounds Information from a Pointer?

- Quite tricky!

```
int *p = (int *) (malloc (k));
…
int *q = p+i;
…
*q = 3; //how to bounds check q?
```

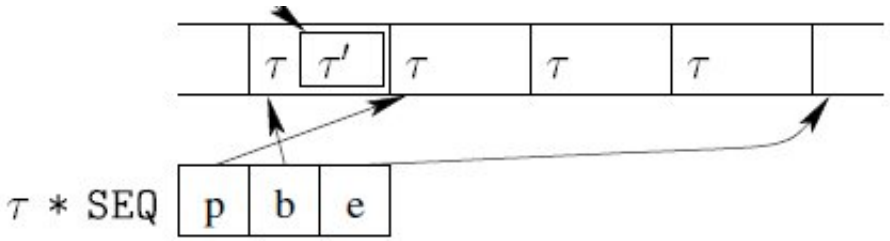
- Idea
  - Dynamically associate bounds information for p at the allocation site
  - Propagate bounds information from p to q
  - Use q's bounds information to check access through q

The Approach of Fat Pointers

- Used in CCured and Cyclone
- Idea: change the representation of pointers to carry bounds information
  - "int *p" becomes

    struct st {int *ptr, int *b, int *e};

    struct st p;
  - the field b points to the beginning of the buffer, and e the end of buffer



Creation of Fat Pointers

- New pointers in C are created in two ways:
  - (1) explicit memory allocation (i.e. malloc()) and
  - (2) taking the address of a global or stack-allocated variable using the '&' operator
- E.g., "p=malloc(size)" becomes

```
p.ptr = malloc(size);
p.b = p.ptr;
p.e = p.ptr+size;
 // to account for possible malloc failure
If (p.ptr == NULL) p.e = NULL;
```

Pointer Copying and Arithmetic

- E.g., “q = p” becomes

  q.ptr = p.ptr;

  q.b = p.b;

  q.e = p.e;

- E.g., “q = p + index” becomes

  q.ptr = p.ptr + index;

  q.b = p.b;

  q.e = p.e;

Example of Instrumented Memory Operations

- "x=*p" becomes

```
if (p.ptr < p.b || p.ptr > p.e) abort();
x= * (p.ptr);
```

Drawbacks of Fat Pointers

- Difficult to interoperate with libraries that do not use fat pointers
  - E.g., char * strchr(const char *s, int C);
    - Need **wrappers** to convert fat pointers to raw pointers and vice versa
  - Need to change the mem layout of data structures
    - E.g., gethostbyname returns the following struct

```
struct hostent {
    char *h name; // String
    char **h aliases; // Array of strings
    int h addrtype;
};
```

    - Every pointer is replaced by three things: pointer, lower bound, and upper bound; the memory layout is changed completely

Drawbacks of Fat Pointers

- Additional work for multi-threaded programs
  - Reading and writing of “int *” is atomic
  - But not for

    struct st {int *ptr, int *b, int *e}
  - Imagine one thread passes a “struct st” variable to another thread
    - The first thread modifies the struct; another thread may see an inconsistent state through its own pointer
  - Need a locking discipline, which may not be there in the original program

SoftBound

- SoftBound
  - Records base and bound information for every pointer as disjoint metadata
  - Check and update such metadata whenever one dereferences and updates a pointer
- Unlike fat pointers, pointer representation in SoftBound remains unchanged
  - Separating metadata from pointers maintains compatibility with C runtime

SoftBound: Creating Pointers

- SoftBound creates metadata for pointers when they are created
  - E.g., “ptr=malloc(size)” becomes

```
ptr = malloc(size);
ptr_base = ptr;
ptr_bound = ptr + size;
if (ptr == NULL) ptr_bound = NULL;
```

  - For a pointer variable, it introduces new vars holding the variable’s bounds

SoftBound: Pointer Arithmetic

- When an expression contains pointer arithmetic (e.g., ptr+index), array indexing (e.g., &(ptr[index])), or pointer assignment (e.g., newptr = ptr;), the resulting pointer inherits the base and bound of the original pointer

```
newptr = ptr + index;     // or &ptr[index]
newptr_base = ptr_base;
newptr_bound = ptr_bound;
```

SoftBound: Retrieving Metadata

- Pointer metadata retrieval
  - SoftBound uses a table data structure to map an address of a pointer in memory to the metadata for that pointer
  - On load

```
int** ptr;
int* new_ptr;
...
check(ptr, ptr_base, ptr_bound, sizeof(*ptr));
newptr = *ptr;       // original load
newptr_base = table_lookup(ptr)->base;
newptr_bound = table_lookup(ptr)->bound;
```

  - On store

```
int** ptr;
int* new_ptr;
...
check(ptr, ptr_base, ptr_bound, sizeof(*ptr));
(*ptr) = new_ptr;      // original store
table_lookup(ptr)->base = newptr_base;
table_lookup(ptr)->bound = newptr_bound;
```

SoftBound

- Downsides
  - Has a significant overhead – 67% for 23 benchmark programs
  - Uses extra memory – 64% to 87% depending on implementation
  - Does not support multithreaded programs
- But, achieve full spatial memory safety for C programs without modifications for benchmarks

Low-Fat Pointers [Kwon et al CCS 13]

- Idea: Hardware support for fat pointers
- Put bases and bounds into 64-bit pointers
  - Use 46 bits for the address and other bits for bounds
  - Hardware instructions to perform desired operations inline
- Result: Memory error protection for 3% overhead

What About Temporal Safety?

- SoftBound + CETS
  - Associate metadata with memory objects (instead of just pointers)
    - When a memory object is deallocated, mark the object invalid through metadata
    - This deals with aliasing well as there maybe multiple pointers that point to the same memory object
      - So updating meta data on the memory object can affect checks for all those pointers (and there is no need to track aliasing)